

机械参数在线辨识代码与文献对应解析

本文档基于文献《基于三角函数正交特性的永磁伺服系统机械参数辨识方法》（吴春）和《Moment of Inertia and Friction Torque Coefficient Identification in a Servo Drive System》（Kim, 2018），对工程中的 `ident.c` 核心辨识代码分别进行了文献公式及理论的对照映射。

以下是详细带有文献对照解释的核心代码：

```
//#####  
// ident.c - 机械参数在线辨识 (吴春正交积分法 + Kim自适应控制框架)  
// F2800157 + 1.5 kHz 速度环  
//#####  
#include "ident.h"  
#include "foc.h"  
#include "user_config.h"  
#include <math.h>  
  
// === 数学函数替换 (原 CMSIS 版本 → 标准 math.h) ===  
#define sin_f32(x)          sinf(x)  
#define cos_f32(x)          cosf(x)  
  
// === 速度环节拍 (与 SPEED_LOOP_EXEC_RATIO 同步) ===  
// 对应文献中离散化积分的时间步长 dt  
#define TS                  (1.0f / SPEED_LOOP_FREQ_Hz)    // ≈ 0.000667  
  
// === 复用工程已有宏 ===  
// KT 为电机转矩常数，对应 Kim 2018 中的  $K_T$ ，用于将电流参考转化为电磁转矩  $T_e = K_T * I_q$   
#define KT                  MOTOR_KT_NM_PER_A              // ≈ 0.04061  
#define MAX_CURRENT_A       MOTOR_MAX_CURRENT_A            // 5.0A  
#define PI_VAL              MATH_PI  
#define RPM_TO_RADPS(rpm)   ((rpm) * (MATH_TWO_PI / 60.0f))  
  
// === 算法收敛参数 ===  
// 对应 Kim 2018 论文 Fig. 12 中的 Low Pass Filter (LPF)  
// 目的是为了防止切入辨识出的  $J_m$  时，速度环 PI 参数突变导致系统震荡（平滑更新）  
#define LPF_ALPHA           0.00209f  
#define MACRO_ITER_MAX      8U          // 大迭代次数：反复辨识以提高精度的次数  
#define A1_SETTLE_CYCLES    4U          // 首轮 A1 缓冲周期：给自适应 PI 控制器留出收敛时间的缓冲圈数  
#define JM_ABSOLUTE_MAX     0.001f      // 防飞车：限制最大合法惯量  
#define ITER_DELAY_TIME     0.5f        // 迭代间歇 (s)  
  
// === 粗测参数 (基于 Kim 2018 Eq(27)) ===  
#define TEST_iq              1.0f        // 粗测注入电流 (A)  
#define TEST_TIME            0.3f        // 粗测时长 (s)  
  
// === 激励频率 ===  
// 对应吴春/Kim文献中的角频率  $\omega_h$  ( $0.5\text{Hz} * 2\pi = \pi \text{ rad/s}$ )  
// 低频可以避免高频噪音，同时确保电流环等效为理想增益为1的环节
```

```

#define W_H          MATH_PI      // 0.5 Hz 机械正弦角频率 (rad/s)

// === 速度环带宽 (用于把 Jm 折算成 PI 增益, Kim 2018 第IV-A节) ===
// 对应 Kim 2018 中的  $\omega_{sc}$  (速度环开环截止频率)
#define W_SC          62.83f      // 10 Hz (20 * PI)
// 对应 Kim 2018 Eq(25) 中的常数 5:  $\omega_{pi} = \omega_{sc} / 5$ , 确保系统相位裕度充足
#define H_RATIO       5.0f        //  $K_i = K_p \cdot W_{sc} / h$ 

//=====
// 全局变量 (供 VOFA / 调试器访问)
//=====
volatile IdentState_e ident_state = STATE_ROUGH_J;
Ident_Output_t      ident_res    = {0};

// 对应吴春文献中两次不同幅值的正弦速度指令 A1, A2
float rpm1          = 300.0f;     // A1 激励幅值 (rpm)
float rpm2          = 600.0f;     // A2 激励幅值 (rpm)
float test_rpm1     = 200.0f;     // 跟踪测试速度 (rpm)

//=====
// 内部状态 (仅 ISR 上下文使用, 加 static)
//=====
static float        dtheta;                //  $d\theta = \omega_h * dt$ , 积分步长
static float        A1, A2;                // 实际激励幅值 (rad/s), 即文献中的 A_1, A_2

static float        time_cnt              = 0.0f;
static float        theta0                = 0.0f; // 积分相位角  $\theta$ 
static float        initial_speed         = 0.0f;

static float        Jm_roughly            = 0.0f; // 粗略惯量 (Kim 2018 Eq 27: Jm_roughly)
static float        Jm_rate               = 0.0f; // 惯量爬坡率 (Kim 2018 Eq 28: Rate of Jm_ini)
static float        Jm_ini                = 0.00001f; // 初始极小惯量 (Kim 2018 中设计的 Jm_ini)

static float        sum_Te_cos            = 0.0f; // 吴春文献中  $\int T_e \cos(\theta) d\theta$  的离散累加器
static float        sum_Te_sin1           = 0.0f; // 吴春文献中 幅值A1下  $\int T_e \sin(\theta) d\theta$  的离散累加器
static float        sum_Te_sin2           = 0.0f; // 吴春文献中 幅值A2下  $\int T_e \sin(\theta) d\theta$  的离散累加器

static float        Jm_target              = 0.0f; // 辨识计算出的目标 Jm
static float        Jm_lpf_out             = 0.0f; // 经低通滤波后的 Jm (Kim Fig. 12)
static uint8_t      is_Jm_switched        = 0U;    // 是否已越过初始爬坡阶段的标志位
static uint16_t     cycle_count            = 0U;    // 当前正弦波已运行的整数圈数

static uint8_t      macro_iter_cnt        = 0U;

```

```

static float Jm_last      = 0.0f;
static float Bm_last      = 0.0f;
static float Cm_last      = 0.0f;

static float Jm_calc      = 0.0f;
static float delta_speed  = 0.0f;

// 速度环 PI 状态 (积分器单位 = A)
static float speed_pi_integral = 0.0f;
static float Iq_ref           = 0.0f;

// PI 增益 (运行时由 Jm_Active 折算)
static float spdPID_Kp_iq;
static float spdPID_Ki_iq;

//=====
// 核心辨识状态机 (每 1.5 kHz 调用一次)
//=====
static void Motor_Parameter_Ident_Process(float omega_fbk, float Te_last)
{
    // 调试器修改 rpm1/rpm2 后实时生效, 转换为 rad/s 对应文献里的 A1 和 A2
    A1 = RPM_TO_RADPS(rpm1);
    A2 = RPM_TO_RADPS(rpm2);

    // 【结合 Kim 2018 Fig. 12 LPF 逻辑】
    // Jm LPF (粗略前用极小的 Jm_ini 让速度环"软启动"爬坡, 成功算出第一个 Jm 后切入一阶
    低通)
    if (ident_state >= STATE_RETURN_ZERO && ident_state <= STATE_EVALUATE)
    {
        if (!is_Jm_switched)
        {
            Jm_lpf_out = Jm_ini;    // 此时系统靠 Jm_ini 驱动, PI 很弱, 等待爬坡
        }
        else
        {
            // Kim 2018 强调: 断续更新 Jm 会导致速度波形严重畸变, 所以必须加低通滤波平
            滑过渡
            Jm_lpf_out = LPF_ALPHA * Jm_target + (1.0f - LPF_ALPHA) * Jm_lpf_out;
        }
    }
    ident_res.Jm_Active = Jm_lpf_out; // 送往速度环控制计算 PI 参数

    switch (ident_state)
    {
        //-----
        case STATE_ROUGH_J:
            // 对应 Kim 2018 Eq(26)-Eq(27): 粗略惯量阶段
            //  $Jm_{roughly} \approx (T2 - T1)/(w2 - w1) * Te$ , 此处  $T1=0$ ,  $w1=0$  简化计算
            if (time_cnt == 0.0f)
            {
                initial_speed = omega_fbk; // 记录 w1
            }
            ident_res.Omega_Ref = 0.0f;    // 开环给定电流, 不使用闭环速度给定
    }
}

```

```

time_cnt += TS; // 累加时间 (对应  $\Delta t$ )

if (time_cnt >= TEST_TIME)
{
    delta_speed = omega_fbk - initial_speed; // 对应 Eq(27) 的分母 ( $w_2$ 
- w1)

    // 对应 Kim Eq(27):  $J_m_{roughly} = (T_e * \Delta t) / \Delta w$ 
    // 其中  $T_e = TEST_{iq} * K_T$ 
    Jm_roughly = (delta_speed > 0.1f)
        ? ((TEST_iq * K_T * TEST_TIME) / delta_speed)
        : JM_ABSOLUTE_MAX;
    if (Jm_roughly > JM_ABSOLUTE_MAX) Jm_roughly = JM_ABSOLUTE_MAX;

    // 对应 Kim 2018 Eq(28): Rate of  $J_m_{ini}$ 
    // 让  $J_m_{ini}$  随时间缓慢上升, 在几个周期内涨到粗测值
    Jm_rate = Jm_roughly / (10.0f * (W_H / (2.0f * PI_VAL)));
    theta0 = 0.0f;
    time_cnt = 0.0f;
    ident_state = STATE_RETURN_ZERO;
}
break;

//-----
case STATE_RETURN_ZERO:
    // 等待电机停转, 准备正式注入正弦波
    ident_res.Omega_Ref = 0.0f; // 速度给 0, 电磁刹车

    if (!is_Jm_switched)
    {
        // Kim 2018:  $J_m_{ini}$  在此阶段开始随着  $J_m_{rate}$  爬坡
        Jm_ini += Jm_rate * TS;
        if (Jm_ini > Jm_roughly) Jm_ini = Jm_roughly;
    }

    if (fabsf(omega_fbk) < 1.0f)
    {
        // 停稳后, 重置积分器, 进入 A1 幅值的正弦激励状态
        theta0 = 0.0f;
        sum_Te_cos = 0.0f;
        sum_Te_sin1 = 0.0f;
        cycle_count = 0U;
        ident_state = STATE_INT_A1;
    }
    break;

//-----
case STATE_INT_A1:
{
    // 给定正弦速度指令: 吴春 Eq(4)  $\omega_1 = A_1 * \sin(\omega_h * t)$ 
    theta0 += dtheta; // 积分步长  $d\theta$ 
    ident_res.Omega_Ref = A1 * sin_f32(theta0);

    // Kim 2018 框架: 如果还在找真实的  $J_m$ , 就继续让  $J_m_{ini}$  爬坡增强 PI 刚度
    if (!is_Jm_switched)

```

项

成的方程1

Jm_ini)

A1_SETTLE_CYCLES : 1U;

```

{
    Jm_ini += Jm_rate * TS;
    if (Jm_ini > Jm_roughly) Jm_ini = Jm_roughly;
}

// 【吴春 2021 核心算法：全周期正交积分法】
// 突破了 Kim 半周期积分只能空载的限制：
// sum_Te_cos: 对应吴春 Eq(13):  $\int Te * \cos(\theta) d\theta$  (一个完整周期  $0 \sim 2\pi$ )
// 根据三角函数正交性，摩擦(sin)与常数负载(TL)乘 cos 积分后均归零，只剩惯量

sum_Te_cos += Te_last * cos_f32(theta0) * dtheta;

// sum_Te_sin1: 对应吴春 Eq(21):  $\int Te1 * \sin(\theta) d\theta$  (一个完整周期  $0 \sim 2\pi$ )
// 根据正交性，惯量(cos)与常值负载(TL)乘 sin 积分归零，分离出 Bm 和 Cm 构

sum_Te_sin1 += Te_last * sin_f32(theta0) * dtheta;

if (theta0 >= 2.0f * PI_VAL) // 完整的一个周期( $2\pi$ )结束
{
    // 对应吴春 Eq(14) 转动惯量辨识结果：
    //  $\hat{J} = [ \int Te * \cos(\theta) d\theta ] / (\pi * A1 * \omega_h)$ 
    Jm_calc = sum_Te_cos / (PI_VAL * A1 * W_H);

    // 首次辨识出合法的 Jm，标志着越过爬坡期 (Kim 2018 Fig.12:  $Jm >$ 
    Jm_ini)

    if (!is_Jm_switched && (Jm_calc > 0.000001f))
    {
        is_Jm_switched = 1U;
        Jm_target = Jm_calc;
    }
    else if (is_Jm_switched && (Jm_calc > 0.000001f))
    {
        Jm_target = Jm_calc;
    }

    cycle_count++;

    // 首轮 A1 缓冲 (A1_SETTLE_CYCLES) 圈
    // 原因: Kim 2018 指出如果速度没跟踪好(PI参数不够)，会产生滞后噪音
    // 必须给自适应 PI 足够的时间，等 Jm 收敛、速度波形纯净后再去切幅值
    uint16_t required_cycles = (macro_iter_cnt == 0U) ?

A1_SETTLE_CYCLES : 1U;
    if (cycle_count >= required_cycles)
    {
        // 满足要求，清零变量，准备送入 A2 幅值
        theta0 = 0.0f;
        sum_Te_cos = 0.0f;
        sum_Te_sin2 = 0.0f;
        ident_state = STATE_INT_A2;
    }
    else
    {
        // 没达到要求圈数，清积分续跑 A1 (抛弃前面不够收敛的周期数据)
        // 切到 A2 时 sum_Te_sin1 保留的一定是最后一圈最纯净的数据

```

```

        theta0      = 0.0f;
        sum_Te_cos   = 0.0f;
        sum_Te_sin1  = 0.0f;
    }
}
break;
}

//-----
case STATE_INT_A2:
    // 给定第二个幅值的速度指令：吴春 Eq(22)  $\omega_2 = A_2 * \sin(\omega_h * t)$ 
    theta0 += dtheta;
    ident_res.Omega_Ref = A2 * sin_f32(theta0);

    // 同样执行全周期正交积分
    sum_Te_cos += Te_last * cos_f32(theta0) * dtheta;

    // sum_Te_sin2: 对应吴春 Eq(23):  $\int Te_2 * \sin(\theta) d\theta$  (分离出方程2)
    sum_Te_sin2 += Te_last * sin_f32(theta0) * dtheta;

    if (theta0 >= 2.0f * PI_VAL) // 第二个幅值的  $2\pi$  结束
    {
        // A2 幅值下同样可以用来校正 Jm
        Jm_calc = sum_Te_cos / (PI_VAL * A2 * W_H);
        if (is_Jm_switched && (Jm_calc > 0.000001f))
        {
            Jm_target = Jm_calc;
        }
        ident_state = STATE_EVALUATE;
    }
    break;

//-----
case STATE_EVALUATE:
{
    ident_res.Omega_Ref = 0.0f;

    // 【吴春 2021 核心算法】：分离粘性摩擦系数(Bm)和库仑摩擦力矩(Cm)
    // 根据吴春文献 Eq(21) 和 Eq(23)，构建关于 Bm 和 Cm 的二元一次方程组：
    // 方程1:  $\text{sum\_Te\_sin1} = B_m * A_1 * \pi + C_m * 4$ 
    // 方程2:  $\text{sum\_Te\_sin2} = B_m * A_2 * \pi + C_m * 4$ 

    // 对应吴春 Eq(24) 解出 Bm_hat:
    //  $B_{\text{hat}} = (\int Te_1 * \sin - \int Te_2 * \sin) / [\pi * (A_1 - A_2)]$ 
    float Bm_calc = (sum_Te_sin1 - sum_Te_sin2) / (PI_VAL * (A1 - A2));

    // 对应吴春 Eq(25) 解出 Cm_hat:
    //  $C_{\text{hat}} = (A_1 * \int Te_2 * \sin - A_2 * \int Te_1 * \sin) / [4 * (A_1 - A_2)]$ 
    float Cm_calc = (A1 * sum_Te_sin2 - A2 * sum_Te_sin1) / (4.0f * (A1 -
A2));

    // 保存辨识结果，并开启前馈补偿
    ident_res.Bm_Active = Bm_calc;
    ident_res.Cm_Active = Cm_calc;
}

```

高精度

```

    ident_res.Enable_FF = 1U;

    macro_iter_cnt++;

    // 多次宏迭代，吴春文献第3节提出："边辨识、边补偿、再辨识"，如此反复能极大提

    if (macro_iter_cnt >= MACRO_ITER_MAX)
    {
        ident_res.Is_Finished = 1U;
        ident_state          = STATE_SUCCESS;
    }
    else
    {
        Jm_last = Jm_target;
        Bm_last = Bm_calc;
        Cm_last = Cm_calc;

        // 清零数据，进入休息态，准备开启下一轮 "边补偿边辨识"
        theta0      = 0.0f;
        sum_Te_cos   = 0.0f;
        sum_Te_sin1  = 0.0f;
        cycle_count  = 0U;
        time_cnt     = 0.0f;
        ident_state  = STATE_ITER_DELAY;
    }
    break;
}

default:
    break;
}
}

//=====
// 速度环入口 (1.5 kHz)
//=====
float Ident_speedLoop_run(float omega_fbk)
{
    // 用上一周期 Iq 还原电磁力矩 (相当于传感器观测的反馈力矩)
    float Te_fbk = Iq_ref * KT;

    Motor_Parameter_Ident_Process(omega_fbk, Te_fbk);

    if (ident_state == STATE_ROUGH_J)
    {
        // 粗测阶段恒定开环加电流，不经过闭环 PI
        Iq_ref = TEST_iq;
    }
    else if (ident_state == STATE_SUCCESS || ident_state == STATE_RESTART)
    {
        Iq_ref          = 0.0f;
        speed_pi_integral = 0.0f;
    }
    else

```

```

{
    // 【Kim 2018 自适应 PI 参数整定模块】(文献第IV-A节)
    // 动态计算速度环 PI 增益 (闭环带宽自整定)

    // 对应 Kim 2018 Eq(24):  $K_{sp} = J_m * \omega_{sc}$ 
    // 此处的  $K_p$  是为了控制  $I_{q\_ref}$ , 所以要把转矩域的  $K_{sp}$  除以转矩常数  $KT$ 
    spdPID_Kp_iq = (ident_res.Jm_Active * W_SC) / KT;

    // 对应 Kim 2018 Eq(25):  $K_{si} = K_{sp} * \omega_{sc} / 5$  (此处  $H\_RATIO = 5.0$ )
    // 设计积分增益保证  $\omega_{pi}$  是  $\omega_{sc}$  的五分之一, 提供足够的相位裕度抑制超调
    spdPID_Ki_iq = (spdPID_Kp_iq * W_SC) / H_RATIO;

    float err = ident_res.Omega_Ref - omega_fbk;

    speed_pi_integral += spdPID_Ki_iq * err * TS;
    if (speed_pi_integral > MAX_CURRENT_A) speed_pi_integral =
MAX_CURRENT_A;
    if (speed_pi_integral < -MAX_CURRENT_A) speed_pi_integral = -
MAX_CURRENT_A;

    float Iq_pi = spdPID_Kp_iq * err + speed_pi_integral;

    // 【Kim 2018 Eq(29) / 吴春 2021 Eq(26) 摩擦前馈补偿】
    // 利用辨识出的粘滞摩擦( $B_m$ )和库仑摩擦( $C_m$ ), 补偿给  $q$  轴电流
    float Iq_ff = 0.0f;
    if (ident_res.Enable_FF)
    {
        // 获取指令速度的符号 sign_w
        float sign_w = (ident_res.Omega_Ref > 0.0f) ? 1.0f
            : (ident_res.Omega_Ref < 0.0f) ? -1.0f : 0.0f;

        // 摩擦转矩模型:  $T_{friction} = B_m * \omega_{ref} + C_m * sign(\omega_{ref})$ 
        float Te_ff = ident_res.Bm_Active * ident_res.Omega_Ref
            + ident_res.Cm_Active * sign_w;

        // 转换为前馈电流:  $Iq_{ff} = T_{friction} / K_T$ 
        Iq_ff = Te_ff / KT;
    }

    // 最终  $I_q$  参考值 = 闭环反馈调节项(PI) + 前馈补偿项(FF)
    // 对应 Kim 2018 Fig. 15 (带有前馈通路的控制结构框图)
    Iq_ref = Iq_pi + Iq_ff;

    if (Iq_ref > MAX_CURRENT_A) Iq_ref = MAX_CURRENT_A;
    if (Iq_ref < -MAX_CURRENT_A) Iq_ref = -MAX_CURRENT_A;
}

return Iq_ref;
}

```